PL-SQL
LABSHEET

NAME: RISHIKA RAJ SUKUMARAN
ROLL NO: AM.SC.U4AIE23146

CLASSWORK

1. Write a function which accepts 2 integer values and return their sum

Query | Query History

```
Create or replace function sum(a int, b int)
returns int as
$$
DECLARE
      c int;
BEGIN
      c:=a + b;
      RETURN c;
END;
$$ LANGUAGE plpgsql;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 335 msec.
```

```
/*** Without declaring C***/
Create or replace function sum_withoutc(a int, b int)
returns int as
$$
BEGIN
     RETURN a + b;
END;
$$ LANGUAGE plpgsql;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 165 msec.
```

Query Query History

```sql
/***Without mentioning vars a and b ***/

Create or replace function sum_positional(int, int)
returns int as
$$
BEGIN
    RETURN $1 + $2;
END;
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 119 msec.
```

```sql
/*** Using alias***/
Create or replace function sum_alias(int, int)
returns int as
$$
DECLARE
    a ALIAS FOR $1;
    b ALIAS FOR $2;
BEGIN
    RETURN a + b;
END;
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 150 msec.
```

```sql
/***IN and OUT parameters***/
Create or replace function sum(a IN int, b IN int, c OUT int)
as
$$
BEGIN
    c := a + b;
END;
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 151 msec.
```

2. Write a function which accepts an int value and must return it as odd/even

```sql
Create or replace function odd_even(a IN int)
returns text as
$$
BEGIN
    IF a%2=0 THEN
        RETURN 'even';
    ELSE
        RETURN 'odd';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Data Output   Messages   Notifications

```
CREATE FUNCTION

Query returned successfully in 111 msec.
```

3. Consider the relation emp(eno, ename, sal). Write a function which accepts an employee no and return the salary of an employee

Query   Query History

```sql
INSERT INTO emp5 VALUES(101, 'Raj', 30000);
INSERT INTO emp5 VALUES(102, 'Megha', 45000);
INSERT INTO emp5 VALUES(103, 'Arun', 50000);

Create or replace function emp_sal(emp_no emp5.eno%TYPE)
returns emp5.sal%type as
$$
DECLARE
    salary emp5.sal%TYPE;
BEGIN
    Select sal into salary from emp5 where eno = emp_no;

    return salary;
END;
$$ LANGUAGE plpgsql;
SELECT emp_sal(102);
```

Data Output   Messages   Notifications

SQL   Sh

	emp_sal integer
1	45000

4. Write a function which accepts a department no and finds the no of employees in that department

Query Query History

```
ALTER TABLE emp5 ADD COLUMN deptno int;
UPDATE emp5 SET deptno = 10 WHERE eno = 101;
UPDATE emp5 SET deptno = 20 WHERE eno = 102;
UPDATE emp5 SET deptno = 10 WHERE eno = 103;
Create or replace function emp_count(dept_no int)
returns int as
$$
DECLARE
    total int;
BEGIN
    Select COUNT(*) into total from emp5 where deptno=dept_no;

    RETURN total;
END;
$$ LANGUAGE plpgsql;
SELECT emp_count(10);
```

Data Output Messages Notifications

Showing rows

	emp_count integer
1	2

5. Write a function which accepts an employee number and gives that employee a 10% hike in salary if the average salary of the corresponding department is greater than the employee’s current salary.

Query Query History

```
Create or replace function hike_update(emp_no emp5.eno%TYPE)
returns void as
$$
DECLARE
    avg_dep    emp5.sal%TYPE;
    emp_sal    emp5.sal%TYPE;
    dep_no     emp5.deptno%TYPE;
BEGIN
    select sal, deptno into emp_sal, dep_no from emp5 where eno = emp_no;
    select avg(sal) into avg_dep from emp5 where deptno = dep_no;
    IF emp_sal<avg_dep THEN
        update emp5 set sal=sal+sal*0.10 where eno=emp_no;
    END IF;
END;
$$ LANGUAGE plpgsql;
SELECT hike_update(103);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No:

	hike_update void
1	

6. Write a function which accepts an employee name and returns the department name of that employee.

```sql
CREATE TABLE Dept5(dep_no INT,dname varchar(25));
INSERT INTO Dept5 VALUES(10, 'HR'),(20, 'Finance'),(30, 'Engineering');
Create or replace function dep_name(emp_name emp5.ename%TYPE)
returns dept5.dname%type as
$$
DECLARE
    dept_name dept5.dname%TYPE;
    dept_no   emp5.deptno%TYPE;
BEGIN
    SELECT deptno INTO dept_no FROM emp5 WHERE ename = emp_name;
    SELECT dname INTO dept_name FROM dept5 WHERE dep_no = dept_no;

    RETURN dept_name;
END;
$$ LANGUAGE plpgsql;
SELECT dep_name('Arun');
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page

	dep_name character varying
1	HR

2. Consider the following schema .
Emp(eno, ename, sal)
EMP_PROJ(eno, pno, prjt_hrs).
a) Write a function ProjectLoad that returns the total project working hours for the given eno.
b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary. New salary is current sal + difference between currrent sal and average salary

Query Query History

```sql
create or replace function ProjectLoad(e INT)
returns INT as
$$
DECLARE
    total INT;
BEGIN
    select SUM(prjt_hrs) into total from emp_proj6 where eno = e;

    IF total IS NULL THEN
        total := 0;
    END IF;

    RETURN total;
END;
$$ LANGUAGE plpgsql;
SELECT ProjectLoad(101);
```

Data Output Messages Notifications

Showing rows: 1 t

	projectload integer
1	25

Query Query History

```sql
create or replace function update_sal(emp_no INT)
returns VOID as
$$
DECLARE
    avg_sal NUMERIC;
    curr_sal NUMERIC;
BEGIN
    select AVG(sal) into avg_sal FROM emp6;
    select sal into curr_sal
    FROM emp6
    WHERE eno=emp_no;

```

Data Output Messages Notifications

SQL Showing rows: 1 to 4

	eno [PK] integer	ename character varying (30)	sal numeric
1	101	Arun	25000
2	102	Riya	30000
3	103	Manoj	18000
4	104	Sneha	27000

Query Query History

```sql

    IF curr_sal<avg_sal then
        UPDATE emp6
        SET sal=sal+(avg_sal-curr_sal)
        WHERE eno=emp_no;
    END IF;
END;
$$ LANGUAGE plpgsql;
SELECT update_sal(101);
SELECT * FROM emp6;
```

Data Output Messages Notifications

SQL Showi

	eno [PK] integer	ename character varying (30)	sal numeric
1	101	Arun	25000
2	102	Riya	30000
3	103	Manoj	18000
4	104	Sneha	27000

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)
Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
create or replace function delete_if_less2(item_no INT)
returns VOID as
$$
DECLARE
    trans_count INT;
BEGIN
    Select COUNT(*) into trans_count from Transaction where ino=item_no;
    IF trans_count<2 then
    Delete from Item where ino=item_no;
    END IF;
END;
$$ LANGUAGE plpgsql;
select delete_if_less2(102);
select delete_if_less2(104);
select * from Item;
```

Data Output Messages Notifications

SQL Showing rows: 1 to 2

	ino [PK] integer	iname character varying (30)	unit_price numeric
1	101	Pen	10
2	103	Pencil	5

4.
Consider a Bank database which includes the following tables.
ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)
a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.

Query Query History

```
create or replace function update_customer_type(threshold NUMERIC, customer_id INT)
returns VOID as
$$
DECLARE
    total_balance NUMERIC;
BEGIN
    select SUM(bal) into total_balance from accounts where cust_no=customer_id;
    IF total_balance is null then
        total_balance:=0;
    END IF;
    IF total_balance>threshold THEN
        update customer6 set c_type='A' where cno=customer_id;
    ELSE
        update customer6 set c_type='B' where cno=customer_id;
    END IF;
END;
$$ LANGUAGE plpgsql;
Select update_customer_type(50000, 201);
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 154 msec.
```

b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```
create or replace function CloseBranch(old_br INT, new_br INT)
returns VOID as
$$
BEGIN
    update accounts set br_no=new_br where br_no=old_br;
    delete from branches where br_no=old_br;
END;
$$ LANGUAGE plpgsql;
select CloseBranch(1, 2);
select * from accounts;
select * from branches;
```

Data Output Messages Notifications

SQL Showing rows: 1 to 2 Page No: 1

	br_no [PK] integer	br_name character varying (30)	loc character varying (30)
1	2	City Branch	Kollam
2	3	Town Branch	Trivandrum

Total rows: 2 Query complete 00:00:00.797

c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

Query Query History

```
create or replace function safe_withdraw(acno INT, amount NUMERIC)
returns TEXT as
$$
DECLARE
    curr_balance NUMERIC;
BEGIN
    select bal into curr_balance from accounts where ac_no=acno;
    IF curr_balance is null then
        return 'Account not found';
    END IF;
    IF curr_balance<amount then
        return 'Insufficient funds';
    ELSE
        update accounts set bal=bal-amount where ac_no=acno;
        return 'Withdrawal successful';
    END IF;
END;
```

Data Output Messages Notifications

SQL

	ac_no [PK] integer	br_no integer	cust_no integer	ac_type character varying (10)	bal numeric

```
$$ LANGUAGE plpgsql;
select safe_withdraw(101, 3000);
select * from accounts where ac_no=101;
```

Data Output Messages Notifications

SQL

	ac_no [PK] integer	br_no integer	cust_no integer	ac_type character varying (10)	bal numeric